Draft Specification for the Sefer Programming Language

Version 0.1.0

Gustek

January 27, 2026

Contents

1 Introduction

A Sefer Torah *(in Hebrew* ספר תורה*, literally "Book of Torah") is a handwritten copy of the Pentateuch in the form of a scroll, used in ritual Torah reading during Jewish prayers. The rules concerning the composition of Torah scrolls are strict: a deficiency in parchment processing, ink properties or a transcription error render the scroll invalid for ritual use.*

Sefer is a purely functional data-processing language. It provides static, polymorphic typing, algebraic datatypes, typeclasses, pattern-matching and a concise yet powerful syntax adapted for both one-liners and more complex scripts. At the core of every Sefer program lies the `Iterator` typeclass and the many iterator adapters defined in its standard library.

Sefer being a purely functional language, no input/output (I/O) access is provided to the user. The only way for a Sefer program to interact with the outside world is to read input from an iterator over the characters (or bytes when binary mode is used) written on the standard input and to return an expression, which will afterwards be displayed on the standard output by the virtual machine when it terminates.

1.1 Rationale

Sefer's main reason for existence is to replace AWK as the go-to utlity for data processing on UNIX-like systems. There are several motivations for this, which are all linked to each other. AWK was designed in the 1970s and its most widely used form (GNU AWK) was published in the late 1980s; AWK is tragically a product of its time, in both its internal and external design. At that time, Pascal and Fortran were mainstream languages and the ANSI C standard was just being finalised. Actually, when AWK was created in 1977, Brian Kernighan (who is also the 'K' in "AWK") and Denis Ritchie's seminal book, *The C Programming Language* (1978), had not even been published yet.

An AWK program is a series of pattern/action pairs, written as `pattern { action }`. The input is split into *records* (by default lines) and the `action` is executed on every record that matches the `pattern`. Two special patterns, namely `BEGIN` and `END` specify actions to be executed before and after the program, respectively. Inside actions, every record can be split into fields (on a given separator) which are accessible through the `$n` variables, where `n` is a number. This means that the entiry of AWK is as powerful as the Sefer combinator `map_cond`, which maps iterator elements using a different function depending on the value of a predicate (see the standard library specification for more information on this adapter). Such lack of expressivity is not too much of a problem on simple one-liners, but quickly grows to become one as task complexity increases.

Even on one liners, AWK suffers from a lack of clarity due to its imperative nature; indeed, declarative programming is especially well-suited for data processing, as the user extracting certain information from the data is more interested in the *result* of the extraction than in its *process*. See for example the following AWK program, which displays the number of words in the input (note that `NF` holds the number of fields in the record):

```
{
  words += NF
}
END { print words }
```

Listing 1.1: `wc -w` as an AWK program

In Sefer, a similar program could be written as follows:

```
||- (words | count)
 | sum
```

Listing 1.2: `wc -w` as a Sefer program

`words` simply constructs an iterator from a string (which is actually but an iterator on characters), splitting it on every whitespace, `count` consumes an iterator and returns the number of elements it yielded and `sum` consumes an iterator, returning the sum of its elements. Another advantage of the declarative paradigm as imposed by Sefer is that the language itself can optimise computation in the backend, while guaranteeing a clear and concise frontend.

AWK is not a bad tool in and of itself, rather a tool that was made for low-resource computers in an era in which the imperative paradigm was quasi-ubiquitous outside of theoretical experiments. AWK is an out-of-date tool that needs a modern replacement and that is what Sefer aims to be.

2 Lexical conventions and syntax

3 Expressions and Types

4 Program structure

A The standard library

B Formal grammar reference